1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA

dei 30 anos Departamento de Engenharia Informática da Universidade de Coimbra

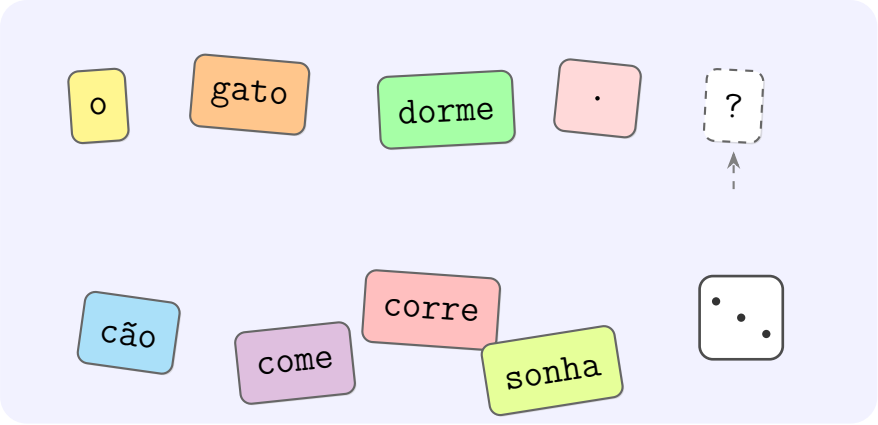
# Introdução à Programação e Resolução de Problemas

LEI

2026-2027

## Mini-Projeto

## Um Modelo de Linguagem Probabilístico




*Que palavra vem a seguir? Atira-se o dado... e o texto escreve-se sozinho!*

# Introdução

Os modelos de linguagem de grande dimensão (os *LLM*, como o ChatGPT ou o Claude) parecem magia, mas partem de uma ideia simples: **dado o texto que já existe, qual é a palavra (ou *token*) mais provável a seguir?** Se soubermos responder a esta pergunta, gerar texto é repetir o processo: prever uma palavra, acrescentá-la ao texto, e voltar a prever.

Neste mini-projeto irá construir, em `Python` , um modelo de linguagem muito mais simples, mas que funciona com a mesma lógica: um **modelo de $n$-gramas**. Em vez de uma rede neuronal com milhares de milhões de parâmetros, o nosso modelo é um *dicionário de contagens* que pode ser impresso e inspecionado. O modelo *lê* um livro, conta que palavras costumam seguir-se a que palavras, e depois usa essas contagens como probabilidades para escrever texto novo com o estilo do livro.

Matematicamente, o modelo estima

$$P(\text{palavra} \mid \text{contexto}) \approx \frac{n(\text{contexto}, \text{palavra})}{n(\text{contexto})},$$

em que $n(\text{contexto}, \text{palavra})$ é o número de vezes que a palavra apareceu a seguir ao contexto no corpus e $n(\text{contexto})$ é o número de vezes que o contexto apareceu. O *contexto* são as últimas $n$ palavras: com $n = 1$ obtemos o modelo de **bigramas**, com $n = 2$ o de **trigramas**, e assim sucessivamente.

## O corpus

Um modelo de linguagem aprende com um **corpus**, isto é, um conjunto grande de texto. Serão utilizados livros em domínio público do Projeto Gutenberg:

- *Amor de Perdição*, de Camilo Castelo Branco (ficheiro de texto com cerca de 310 kB):
  `https://www.gutenberg.org/ebooks/16425`
  Escolha o formato **Plain Text UTF-8** e guarde o ficheiro como `amor_de_perdicao.txt`.

- *O Crime do Padre Amaro*, de Eça de Queirós (cerca de 877 kB), opcional, necessário apenas para uma das extensões do projeto final:
  `https://www.gutenberg.org/ebooks/31971`
  Guarde-o como `crime_padre_amaro.txt`.

Estes textos usam a ortografia da época (por exemplo, *fóra*, *quasi*, *elle*). Isso não é um problema: o modelo aprende apenas aquilo que lá está, e o texto gerado vai ter esse sabor arcaico. Cada ficheiro tem, no início e no fim, texto do Projeto Gutenberg (licença, créditos) que não faz parte do livro; o primeiro exercício trata disso.

## Regras e convenções

- Apenas pode usar o `Python` padrão e os módulos `random` e `math`.
- Todas as funções que escrever ao longo do projeto formam um único programa. Cada exercício, no entanto, indica o que fazem as funções de que depende, para que possa ser resolvido isoladamente.
- Um **token** é uma palavra em minúsculas ou um dos sinais de fim de frase `.`, `!` e `?`. Ao longo da ficha usamos o tuplo

  ```
  FIM = (".", "!", "?")
  ```

- Durante o desenvolvimento, teste tudo num **corpus de brincar** (poucas frases) antes de o executar no livro inteiro. Os exemplos do enunciado foram pensados para isso.
- Quando um exercício pede um resultado aleatório, o seu resultado será diferente do de um colega, e isso é normal. Se necessitar de resultados reprodutíveis durante a resolução, utilize `random.seed(1)` no início do programa.

## Organização

O trabalho está dividido em quatro secções, que constroem o modelo de forma progressiva: (1) preparação e análise do texto; (2) modelo de bigramas e geração de texto; (3) modelos de $n$-gramas; (4) temperatura e texto final.

# 1 Do texto aos números

Nesta secção, um livro é transformado numa lista de *tokens* e são realizadas as primeiras contagens. Ainda não existem probabilidades nem geração de texto, mas já é possível descobrir propriedades curiosas da língua.

**Problema 1.1**

**Ler e limpar o livro.**

(a) Escreva a função `ler_linhas(nome)`, que abre o ficheiro `nome` (codificação UTF-8) e devolve uma lista com as suas linhas, sem o carácter de fim de linha.

(b) Os ficheiros do Projeto Gutenberg têm texto que não pertence ao livro. O livro começa na linha a seguir àquela que começa por `*** START OF` e termina na linha anterior àquela que começa por `*** END OF`. Escreva a função `extrair_corpo(linhas)` que devolve apenas as linhas do livro. Se os marcadores não existirem, devolve todas as linhas.

(c) Imprima o número de linhas e o número total de caracteres do livro.

```
>>> extrair_corpo(["Título", "*** START OF THE PROJECT
   GUTENBERG EBOOK X ***",
...                "linha 1", "linha 2",
...                "*** END OF THE PROJECT GUTENBERG
   EBOOK X ***", "licença"])
['linha 1', 'linha 2']
```

■

**Problema 1.2**

**Tokenizar.** Escreva a função `tokenizar(texto)`, que recebe uma *string* e devolve a lista dos seus tokens, segundo as regras:

- Todo o texto é convertido para minúsculas.
- Uma palavra é uma sequência de letras. Um hífen ou um apóstrofo *no meio* de uma palavra fazem parte dela (`disse-lhe`, `d'aquella` são um só token).
- Os sinais `.`, `!` e `?` são tokens por si só. Se aparecerem vários seguidos (por exemplo, reticências), só o primeiro conta.

- Qualquer outro carácter (espaços, vírgulas, dígitos, travessões, parênteses, ...) apenas separa palavras e é descartado.

**Sugestão:** percorra o texto carácter a carácter, acumulando as letras da palavra atual numa *string*; o método `str.isalpha()` diz se um carácter é uma letra.

```
>>> tokenizar("O gato dorme. O cão d'aquella casa não
    come!")
['o', 'gato', 'dorme', '.', 'o', 'cão', "d'aquella", '
    casa', 'não', 'come', '!']
>>> tokenizar("Olá... tudo bem?")
['olá', '.', 'tudo', 'bem', '?']
>>> tokenizar("-- Sim, senhor! [1]")
['sim', 'senhor', '!']
```

Aplique a função ao livro inteiro (junte primeiro as linhas numa só *string* com `.join(linhas)`) e imprima o número de tokens.

■

## Problema 1.3

**Contar palavras.** Escreva a função `contar_frequencias(tokens)` que recebe uma lista de tokens e devolve um dicionário que associa cada token ao número de vezes que aparece.

```
>>> contar_frequencias(["o", "gato", "come", "o", "rato"
    ])
{'o': 2, 'gato': 1, 'come': 1, 'rato': 1}
```

■

## Problema 1.4

**As palavras mais frequentes.** Escreva a função `top_n(freqs, n)` que recebe um dicionário de frequências e devolve uma lista com os $n$ pares `(palavra, frequência)` de maior frequência, por ordem decrescente. Para treinar ciclos, **não utilize** `sorted` nem `max`: encontre o maior elemento com um ciclo, guarde-o, retire-o de uma *cópia* do dicionário (o original não pode ser alterado) e repita. Em caso de empate, fica primeiro o que apareceu primeiro no dicionário. Se o dicionário tiver menos de $n$ elementos, devolva todos.

```
>>> top_n({"o": 5, "a": 3, "de": 4, "que": 1}, 2)
[('o', 5), ('de', 4)]
```

Imprima as 10 palavras mais frequentes do livro. Que tipo de palavras domina a lista?

## 2 Bigramas e geração de texto

Nesta secção, o computador aprende a escrever. Um **modelo de bigramas** guarda, para cada palavra, **todas as palavras que alguma vez a seguiram** no corpus e o número de vezes que cada uma a seguiu. É um dicionário de dicionários:

```
corpo = ["o", "gato", "dorme", ".", "o", "gato", "come",
   ".", "o", "cão", "dorme", "."]
```

Este **corpus de brincar** será utilizado nos exemplos desta secção.

**Problema 2.1**

**Construir o modelo.** Escreva a função `construir_bigramas(tokens)` que devolve um dicionário em que:

- cada **chave** é um token do corpus;
- o **valor** é outro dicionário que contém **todos os tokens diferentes que seguiram essa chave**, em qualquer ponto do corpus, cada um associado ao número de vezes que apareceu imediatamente a seguir a ela.

Por exemplo, no corpus de brincar, a palavra `"o"` aparece três vezes: duas seguida de `"gato"` e uma seguida de `"cão"`. Por isso, o valor associado à chave `"o"` é `{"gato": 2, "cão": 1}` e *não* apenas o primeiro (ou o último) token que a seguiu.

```
>>> construir_bigramas(["o", "gato", "dorme", ".", "o", "
   gato", "come", ".",
...                      "o", "cão", "dorme", "."])
{'o': {'gato': 2, 'cão': 1}, 'gato': {'dorme': 1, 'come':
    1},
 'dorme': {'.': 2}, '.': {'o': 2}, 'come': {'.': 1}, 'cão
    ': {'dorme': 1}}
```

Repare que o último `"."` não tem sucessor, por isso não contribui para o dicionário. Construa o modelo do livro e imprima o dicionário associado a uma palavra à sua escolha (por exemplo, `"mar"`).

■

### Problema 2.2

**Probabilidade de uma palavra.** Escreva a função `probabilidade(modelo, a, b)` que devolve $P(b \mid a)$, isto é, a fração das vezes em que a palavra `a` foi seguida de `b` no corpus. Devolva `0.0` se `a` não existir no modelo ou nunca tiver sido seguida de `b`.

```
>>> probabilidade(modelo, "o", "gato")      # modelo do
   corpus de brincar
0.6666666666666666
>>> probabilidade(modelo, "o", "rato")
0.0
```

Calcule, no livro, a probabilidade de a palavra `"o"` ser seguida de `"mar"`.

■

### Problema 2.3

**Escolher ao acaso, com pesos.** Escreva a função `escolher_proxima(sucessores)` que recebe um dicionário de contagens (por exemplo, `{"gato": 3, "cão": 1}`) e devolve **uma** palavra, escolhida ao acaso com probabilidade proporcional à contagem: neste exemplo, `"gato"` com 75% e `"cão"` com 25%.

Implemente-a **à mão**, sem `random.choices`, com o método da roleta:

1. some todas as contagens (`total`);
2. sorteie um inteiro `r` entre 1 e `total` com `random.randint(1, total)`;
3. percorra as palavras acumulando as contagens e devolva a primeira cujo acumulado seja maior ou igual a `r`.

Para verificar, chame a função 10 000 vezes com `{"gato": 3, "cão": 1}` e conte quantas vezes saiu `"gato"`: deve obter um valor próximo de 7500.

■

### Problema 2.4

**Gerar uma frase.** Escreva a função `gerar_frase(modelo, max_tokens=30)` que gera uma frase:

1. a primeira palavra é escolhida entre as que, no corpus, aparecem a seguir a `"."` (utilize `escolher_proxima` com `modelo["."]`);

2. enquanto não se verificar nenhuma das condições de paragem, escolha a palavra seguinte a partir dos sucessores da palavra atual;

3. pare quando o último token for um sinal de `FIM`, ou a frase já tiver `max_tokens` tokens, ou a palavra atual não tiver sucessores no modelo.

A função devolve a lista de tokens da frase. Gere e imprima 5 frases do livro (ainda sob a forma de listas de tokens).

■

**Problema 2.5**

**Escrever um parágrafo.** A lista de tokens ainda não parece texto. Escreva:

(a) `formatar(tokens)`, que junta os tokens numa *string* legível: palavras separadas por um espaço, sinais de `FIM` colados à palavra anterior, e a primeira palavra de cada frase com inicial maiúscula (`str.capitalize()`).

(b) `gerar_texto(modelo, n_frases)`, que gera `n_frases` frases e as devolve formatadas numa única *string*.

```
>>> formatar(["o", "gato", "dorme", ".", "o", "cão", "
    come", "!"])
'O gato dorme. O cão come!'
```

Gere um parágrafo com 5 frases a partir do livro. Leia-o. As frases são razoáveis *localmente* (duas palavras seguidas fazem sentido)? E *globalmente*? O que é que o modelo não consegue "saber"?

■

# 3 Modelos de $n$-gramas

E se o modelo olhar para **mais do que uma palavra** de contexto? Num modelo de ordem $n$, a chave do dicionário é um **tuplo** com as $n$ palavras anteriores (os tuplos, ao contrário das listas, podem ser chaves de dicionários). Com $n = 2$ e o corpus de brincar da secção anterior, o contexto `("o", "gato")` é seguido ora por `"dorme"`, ora por `"come"`.

## Problema 3.1

**Construir o modelo de ordem $n$.** Escreva a função `construir_modelo(tokens, n)` que devolve um dicionário em que cada chave é um tuplo com $n$ tokens consecutivos e o valor é o dicionário dos tokens que se seguem a esse contexto, com as contagens. Usa fatias de listas para obter os contextos.

```
>>> m = construir_modelo(["o", "gato", "dorme", ".", "o",
    "gato", "come", ".",
...                        "o", "cão", "dorme", "."], 2)
>>> len(m)
8
>>> m[("o", "gato")]
{'dorme': 1, 'come': 1}
>>> m[(".", "o")]
{'gato': 1, 'cão': 1}
```

Com $n = 1$ obtém-se exatamente o modelo de bigramas da secção anterior, mas com as chaves em tuplos de um elemento, por exemplo `("o",)`.

■

## Problema 3.2

**Gerar com contexto deslizante.** Assuma que dispõe de `escolher_proxima(sucessores)`. Escreva a função `gerar(modelo, max_tokens=40)` que gera uma frase com um modelo de qualquer ordem $n$ (o valor de $n$ deduz-se do tamanho das chaves):

1. o **contexto inicial** é um contexto do modelo cujo primeiro elemento é `"."` e que não contém mais nenhum sinal de `FIM`; escolha um ao acaso (`random.choice` sobre uma lista dos contextos possíveis). A frase começa com os restantes elementos desse contexto (todos menos o `"."`);

2. em cada passo, escolha o token seguinte a partir de `modelo[contexto]`, junte-o à frase e **deslize o contexto**: retire o token mais antigo e acrescente o novo (`contexto = contexto[1:] + (novo,)`);

3. pára quando sair um sinal de `FIM`, quando se atingir `max_tokens`, ou se o contexto não existir no modelo.

Teste automaticamente: para $n = 1, 2, 3$, gere 100 frases e verifique que, para cada uma, todas as janelas de $n + 1$ tokens consecutivos de `["."] +`

`frase` existem no modelo (o contexto de $n$ tokens existe e o token seguinte é um dos seus sucessores).

■

**Problema 3.3**

**Gerar texto com $n$-gramas.** Assuma que dispõe das funções `gerar` e `formatar`. Escreva a função `gerar_texto_n(modelo, n_frases)`, que gera `n_frases` frases com um modelo de qualquer ordem e as devolve formatadas numa única *string*.

Para $n = 1, 2, 3, 4$, construa o modelo do livro e gere um parágrafo de 5 frases. Compare os quatro textos:

(a) O que muda à medida que $n$ aumenta? Para que valor de $n$ o texto lhe parece melhor?

(b) Escolha algumas expressões do texto gerado com $n$ elevado e procure-as no ficheiro do livro. O modelo está a inventar ou a copiar? Porquê?

■

# 4 Temperatura e texto final

Nesta secção, a "criatividade" do modelo passa a ser controlada por um parâmetro e todo o trabalho anterior é reunido na produção de um texto original.

**Problema 4.1**

**Temperatura.** Os LLM têm um parâmetro chamado *temperatura*, que controla a "criatividade". Também o podemos ter no nosso modelo. Dado um dicionário de contagens, a probabilidade da palavra $w$ com temperatura $T > 0$ é

$$P_T(w) = \frac{\text{contagem}(w)^{1/T}}{\sum_u \text{contagem}(u)^{1/T}}.$$

Com $T = 1$ obtemos as probabilidades normais; com $T < 1$ as palavras mais prováveis ficam ainda mais prováveis; com $T > 1$ a distribuição aproxima-se da uniforme.

(a) Escreva `distribuicao_temp(sucessores, T)`, que devolve um dicionário palavra → probabilidade.

(b) Escreva `escolher_proxima_temp(sucessores, T)`, que sorteia uma palavra segundo essa distribuição (utilize `random.random()`, que devolve um decimal entre 0 e 1, e o método da roleta com probabilidades).

(c) Assumindo que dispõe de `gerar` e `escolher_proxima`, escreva `gerar_temp(modelo, T, max_tokens=40)`, uma versão de `gerar` (para qualquer ordem) que usa a temperatura.

```
>>> distribuicao_temp({"a": 8, "b": 2}, 1)
{'a': 0.8, 'b': 0.2}
>>> distribuicao_temp({"a": 8, "b": 2}, 0.5)   # mais
   conservador
{'a': 0.9411764705882353, 'b': 0.058823529411764705}
>>> distribuicao_temp({"a": 8, "b": 2}, 2)     # mais
   aleatório
{'a': 0.6666666666666666, 'b': 0.3333333333333333}
```

(d) Gere 3 frases do livro com $T = 0{,}3$, $T = 1$ e $T = 2$ (modelo de ordem 2). Descreva a diferença. O que acontece quando $T \to 0$? E quando $T \to \infty$?

■

**Problema 4.2**

**O seu texto.** Reúna tudo o que construiu para produzir um texto original e guardá-lo num ficheiro.

(a) Escreva a função `guardar_texto(texto, nome)`, que escreve a *string* `texto` no ficheiro `nome` (codificação UTF-8).

(b) Escreva um programa que lê o livro, constrói um modelo de ordem $n$ à sua escolha, gera um texto com pelo menos 10 frases (com a temperatura que preferir, usando `gerar_temp` e `formatar`) e o guarda no ficheiro `meu_texto.txt`.

(c) Experimente diferentes valores de $n$ e de temperatura até obter um texto de que goste. Acrescente no fim do ficheiro uma linha de comentário com os parâmetros utilizados e uma apreciação do resultado (coerência, originalidade).

**Extensões (opcionais):**

- **Modelo de caracteres:** em vez de palavras, utilize caracteres como tokens. Compare o texto gerado com o do modelo de palavras.
- **Misturar autores:** treine um modelo com os dois livros (o segundo é *O Crime do Padre Amaro*) e gere texto "entre Camilo e Eça".
- **Autocompletar:** um programa que lê uma frase do utilizador (`input()`) e sugere as 3 palavras seguintes mais prováveis, com as respetivas probabilidades.

■

# 5 Datas e Modo de Entrega

O trabalho deverá ser realizado parcialmente durante as aulas e por grupos com uma dimensão máxima de 2 alunos. Irá existir uma defesa obrigatória, em que todos os elementos do grupo têm de estar presentes.

# 6 Datas e Modo de Entrega

O trabalho deverá ser realizado parcialmente durante as aulas e por grupos com uma dimensão máxima de 2 alunos. Irá existir uma defesa obrigatória, em que todos os elementos do grupo têm de estar presentes.

### Modo de Entrega:

Todos os elementos (por exemplo, código ou ficheiros de texto) produzidos durante o trabalho deverão ser entregues eletronicamente através do Inforestudante, num único ficheiro zip.

Todos os ficheiros Python entregues deverão começar por um cabeçalho em comentário que identifique a disciplina, o mini-projeto e o número de estudante de todos os elementos do grupo, tal como no exemplo seguinte:

```
# ---------------------------------------------
# Introdução à Programação e Resolução de Problemas
# Mini-Projeto: Um Modelo de Linguagem Probabilístico
#
# Grupo:
#   Número de estudante do aluno 1
#   Número de estudante do aluno 2
# ---------------------------------------------
```

**Data Limite: 23h59 do dia 04 de Dezembro de 2026**

# 7 Considerações finais

Ao longo do projeto, tenha presentes os seguintes princípios:

- **Trabalhe de forma progressiva.** O projeto está dividido em problemas que se complementam. Resolva cada problema e teste a sua solução antes de avançar para o seguinte.
- **Pense antes de programar.** Compreenda o problema e defina uma estratégia de resolução antes de escrever código.
- **Comece com exemplos pequenos.** Teste as suas funções com dados simples antes de as aplicar ao livro completo.
- **Aprenda com os erros.** Identificar e corrigir problemas faz parte do processo de aprendizagem.
- **Privilegie soluções simples e claras.** Escreva código que compreenda e que seja capaz de explicar.
- **Compreenda o trabalho realizado.** Todos os elementos do grupo devem conhecer as soluções desenvolvidas e ser capazes de as explicar na defesa.

O mais importante não é apenas conseguir que o programa funcione, mas compreender *por que razão* funciona.

Nuno Lourenço, Tiago Baptista, João Correia, Carlos Fonseca, Luís Gonçalo, Gonçalo Lopes, João Macedo, Maria José Marcelino, Telmo Neves, Marco Silva, Teresa Salazar – 2026/2027